DeepSeek Harness Handbook • dsh-handbook

> ***From zero to one with DeepSeek Harness — the beginner's encyclopedia for DeepSeek's open-source agent runtime.*** *中文 · English · Live-updating with dsh rc releases*

What this is

DeepSeek Harness (`dsh`) is the agent runtime open-sourced by DeepSeek on 2026-08-13 — an "everything is a plugin" framework. Official docs focus on architecture; **this handbook fills the beginner path**: from "what is an agent runtime" to install, usage, plugin development and performance tuning — every command copy-pasteable and verified on real hardware. **Any developer can go from zero to productive.**

30-second quickstart

```
# 1. Install (needs Node.js ≥ 22)
npx -y @deepseek-ai/dsh web

# 2. Open http://127.0.0.1:3080 and chat
# 3. Or run a one-shot task (scripts / CI)
dsh --profile headless "Hello, introduce yourself in one sentence"
```

Table of contents (zero → one)

#	Chapter	What you'll learn	Status
1	Understanding DeepSeek Harness	What it is, why it matters, vs Claude Code	✅ EN
2	5-Minute Quickstart	Install, web/headless modes, models & reasoning effort, troubleshooting	✅ EN
3	Profiles & the Plugin System	The customizable skeleton, mounting plugins, host/client halves, extension points, real pitfalls	✅ EN
4	Plugin Development, Hands-On	Write your first plugin (full code + tests + live verification)	✅ EN
5	Real-World Cases	Three real open-source PRs, end to end	✅ EN
6	Advanced & Performance Tuning	reasoning_effort strategy, latency analysis, pitfalls	✅ EN
7	Ecosystem & Resources	Official entry points, how to join, reading paths	✅ EN

8	Tools & Context System	60+ capability packages, built-in tools, context injection, compaction, security model	✅ EN
9	MCP, Subagents & Workflows	External tool integration, parallel subagents, multi-step orchestration, agent system blueprint	✅ EN
10	Complex Real-World Cases	**Run live in dsh**: data cleaning + visualization pipeline (186s), 5-bug fix + 49 tests (94s), with artifact & judgment analysis	✅ EN

Demo

Demo	What it shows
📺 30s overview of dsh	Install → Web UI → Headless → plugins (real screenshots)
⚡ Community plugin demo (planned)	Git panel / tool-turbo hands-on

PDF

- **Chinese full edition**: DeepSeek-Harness-白皮书.pdf (7 chapters, 1.25MB)
- English PDF will be published as English chapters land

Why read this (vs official docs)

Official docs	This handbook
Architecture view (AGENTS.md / architecture.md)	**Beginner view**: a zero-to-one path
Scattered examples	**Every chapter runnable**, commands verified
English only	**Bilingual**, Chinese-first
No ecosystem practice	**Real plugin/PR breakdowns** (pitfalls & safety included)

Ecosystem links

Methodology comes from real open-source work:

- dsh-tool-turbo — tool-call latency optimizer (source of chapters 4/6)
- DSH-better-sidebar — community sidebar plugin (chapter 5 cases)

Contribute

- Commands broken? rc releases iterate fast — open an issue
- Want to help? See Chapter 7: Ecosystem

Chapter 1: Understanding DeepSeek Harness

> *Goal: build a complete mental model of DeepSeek Harness (dsh) from absolute zero — **what it is, why it matters, how it differs from mainstream agents, and when to use it**. No prior knowledge required.*

1.1 Three intuitions first

① What is dsh? — "the LEGO baseplate for agents." LEGO gives you the baseplate and standard bricks (runtime + core plugins); you assemble freely (add plugins, swap UIs, change behavior). Claude Code, by contrast, is "a finished car" — great to drive, but modifying the engine means asking the vendor.

② Why "harness"? — the engineering layer around a model. A model (DeepSeek V4) only "replies with text." To make it work in your repository (read files, run commands, edit code, loop), you need an engineering layer on top: session management, tool calling, context control, error recovery. **That layer is a harness.** dsh is DeepSeek open-sourcing that layer.

③ Why open-source it now? — agents entered the "programmable era". 2025 was model-capability competition; 2026 is agent-engineering competition. Open-sourcing the harness is the strategic move to make "how to organize agents" an open ecosystem — like Android did for phones.

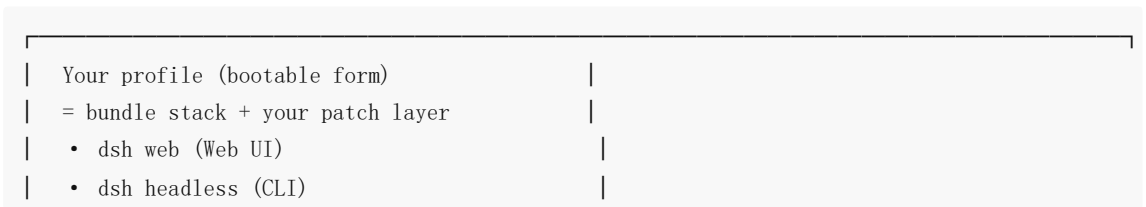
1.2 Official facts

One-liner: DeepSeek Harness (`dsh`) is DeepSeek's open-source agent runtime with an "everything is a plugin" architecture, built on the Cordis plugin container.

Fact	Value
Open-sourced	2026-08-13
License	MIT
Language	TypeScript (Node.js ≥ 22)
Version	`0.1.0-rc.x` (rc.6; fast iteration, breaking changes expected)
Runtime	Cordis
Built-in profiles	`web` + `headless`

1.3 How "everything is a plugin" works

Layered view



```
|   • your custom profile (TUI/desktop/bot…)     |
├──────────────────────────────────────────────────┤
|  Capability layer (each = a plugin)            |
|   • llm • tools • session • client • settings  |
|  … (60+ official packages)                     |
├──────────────────────────────────────────────────┤
|  Cordis container: loading, DI, events, lifecycle |
└──────────────────────────────────────────────────┘
```

Three concepts you must know

① **Profile (bootable form)** — a directory `$DSH_HOME/profiles/<name>/` with `package.json` (plugin deps + manifest) and `cordis.patch.yml` (your patch layer). Load order: built-in bundles → profile patch → global patch → `--patch` overlays.

② **host half / client half** — one npm package can carry both: the host half (`apply(ctx)` on Node: tools, services, events) and the client half (browser UI, declared via `dsh.client` in package.json).

③ **Extension points** — official principle: *"Plugins, not loop changes."* Use hooks, don't fork the core. Common ones: `agent/request` waterfall (change request config per step), `conversationEvents.register` , `ctx.slots.inject` (inject UI), `settings` service.

1.4 dsh vs mainstream agents

Capability matrix

Dimension	dsh	Claude Code	OpenAI Codex	OpenCode	Gemini CLI	Kim CLI
Open source	✅ MIT	❌	❌	✅ MIT	❌	❌
Model binding	model-agnostic	Claude	GPT	any	Gemini	Kimi
Official runtime	✅ + plugin ecosystem	product	product	client only	product	produ
Plugin system	**first-class, 60+ official pkgs**	config/hooks	config	config	none	none
Custom UI	✅ (client half)	❌	❌	partial	❌	❌
Automation/CI	✅ headless	✅	✅	✅	✅	✅
TUI	plugin-provided	✅ built-in	✅ built-in	✅ built-in	✅	✅
Ecosystem stage	day-zero (2026-08-13)	mature	mature	mature	mature	early

Best for	**deep customization + ecosystem**	out-of-box	out-of-box	OpenCode users	Google	Kimi

Case: same task, different doors

Task: "Find every call of function X in the repo and change one argument."

Agent	How you do it
dsh (web)	`dsh web` → type → model uses Grep/Read/Edit tools; add plugin sidebars (Git panel, tool-turbo)
dsh (headless)	`dsh --profile headless "task"` → prints result, exits — **CI-friendly**
Claude Code / Codex / OpenCode / Kimi	open the TUI → type → model does it

The difference: same sentence, but dsh gives you a choice dimension — swap the UI and toolchain. Others ship a fixed interface.

Case: real workloads (our measurements, 2026-08-13)

Scenario	dsh measured	Note
Simple file create	cold start ~110s (first round, context injection) → hot cache ~1s	reasoning effort is the main variable
50-step tool chain	LLM 10m+, tool calls 9m+	per-step thinking accumulates — **that's the speed plugin's value**
Plugin dev	zero to runnable plugin: 1 day (tests + live verify)	clear extension points
vs Claude Code same task	dsh + V4-Flash ≈ 1/10–1/30 of Claude cost	price dimension favors dsh ecosystem

1.5 When to use dsh

✅ **Adopt now**: model-agnostic + UI-optional + behavior-mutable base; be an early ecosystem contributor; run agents on servers/CI; cost-sensitive.

⏸ **Wait**: you need an out-of-the-box coding assistant; can't accept rc breaking changes; deeply dependent on a vendor's exclusive model feature.

1.6 FAQ

- **Is dsh a model?** No. It's a runtime; models plug in via the `llm` plugin (DeepSeek V4 native, OpenAI-compatible others possible).

- **dsh vs OpenCode?** Both open-source agent clients; OpenCode is "client + config", dsh is "runtime + official plugin ecosystem" (official bundles, 60+ packages).
- **Do I need TypeScript?** To *use* it, no. To *write plugins* (Ch.4), basic TS — with full code in the handbook.
- **Is it stable?** rc stage; use for ecosystem/tinkering now, production core after 0.1.0.
- **Why learn dsh now?** Day-zero ecosystem + official encouragement + Chinese-tutorial vacuum — first-mover window.

Next: Chapter 2: 5-Minute Quickstart

Chapter 2: 5-Minute Quickstart

> *Goal: **follow along and get it running.** Every command shows its expected output and common errors. Open a terminal and go.*

2.1 Prerequisites (30-second check)

Need	Check	Pass
Node.js ≥ 22	`node --version`	`v22.x+` (24 recommended)
npm	`npm --version`	prints a version
Network	npm registry reachable	can install packages
(optional) DeepSeek API key	https://platform.deepseek.com	for real conversations

> *dsh starts without a key (UI opens), but conversations need one. Examples assume it's configured.*

2.2 Install (two ways)

One-liner (recommended for new users)

```
npx -y @deepseek-ai/dsh --version
```

First run downloads dsh (large package, 40+ plugin modules, 1-3 min). You're good when you see:

```
0.1.0-rc.6
```

Global install (recommended for frequent use)

```
npm install -g @deepseek-ai/dsh
dsh --version
```

2.3 Mode 1: Web UI (`dsh web`)

Start

```
dsh web
```

Expected output:

```
dsh web: http://127.0.0.1:3080
```

Open http://127.0.0.1:3080 in your browser.

UI map (see screenshot)

Area	What's there
Left	session list / workspace switch / new session
Center	chat: input box, model selector (`DeepSeek V4 Flash`), reasoning level (`High`)
Right/bottom	plugin sidebar (empty by default; appears after installing community plugins)
Top-right	Session log / Trajectory

First conversation

1. Click **New session**
2. Type: `Hello, introduce yourself in one sentence`
3. Press Enter

Expected reply (roughly):

> *Hello! I'm a DeepSeek-powered AI coding assistant...*

Models & reasoning effort

Click the model selector next to the input:

Model	Positioning
`deepseek-v4-flash` (default)	value: fast, cheap, enough for daily work
`deepseek-v4-pro`	flagship: stronger, more expensive/slower

Reasoning effort (three steps since 2026-08-13):

Level	Speed	Quality	Suggested use
`low`	fastest	adequate	simple/deterministic rounds, batch, cheap chain steps
`high` (default)	medium	good	daily agent tasks
`max`	slowest	best	hard reasoning, long planning

> 💡 ***Key performance insight**: the model re-thinks **before every tool call**. Measured: a "create file" task spends ~90% of wall-clock in thinking; a 50-step tool chain accumulates minutes.*

> ***Lowering the reasoning effort is the highest-leverage speedup*** *(Ch.6 + the* `dsh-tool-turbo` *plugin).*

2.4 Mode 2: Headless (one-shot tasks, scripts/CI)

```
dsh --profile headless "Hello, introduce yourself in one sentence"
```

Expected output (prints result, process exits):

```
Hello! I'm a DeepSeek-powered AI coding assistant...
```

Headless value:

- **Automation**: CI, servers, cron
- **Script-friendly**: non-zero exit = failure; output pipes
- **Isolation**: fresh session per call (`--resume` restores; see `dsh --profile headless --help`)

Example: daily digest script:

```
dsh --profile headless "Read today's git log in the workspace and write a Chinese daily
summary" > daily-report.md
echo "exit=$?"
```

2.5 Your first plugin: a Git panel for web

dsh's sidebar is empty by default — install community plugin `dsh-better-sidebar` to feel "everything is a plugin" (mechanics in Ch.3; here just get it running):

```
# 1. locate your web profile
#    Windows: %USERPROFILE%\.dsh\profiles\web
#    macOS/Linux: ~/.dsh/profiles/web

# 2. add one line to package.json dependencies
#    "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"

# 3. add to cordis.patch.yml
#    - insert:
#        - id: better-sidebar
#          name: dsh-better-sidebar

# 4. install & restart
cd ~/.dsh/profiles/web && pnpm install
dsh web
```

After restart, the sidebar gains file manager / terminal / **Git panel** / browser tabs.

> *The panel's fetch/pull/push buttons are a community PR (Ch.5) — **this is how the plugin ecosystem works.***

2.6 Config & paths

First run creates:

```
~/.dsh/
├─── settings.yaml          # global settings (model, reasoning effort)
├─── profiles/              # profile dirs
|    └─── web/
|         ├─── package.json      # plugin deps + manifest
|         └─── cordis.patch.yml  # your patch layer
├─── sessions/              # session data
└─── storages/              # persistence
```

`settings.yaml` example:

```yaml
agent-default-model:
  model: deepseek-v4-flash
  reasoningEffort: high
```

2.7 Command cheatsheet

Command	Purpose
`dsh web`	start Web UI (alias `dsh --profile web`)
`dsh --profile headless "task"`	one-shot task, print result, exit
`dsh plugin --profile <name> add <pkg>`	add a plugin to a profile
`dsh --dump-config`	print composed config tree
`dsh --profile tui`	TUI mode (plugin-provided; not built-in)
`dsh --version`	version

2.8 Troubleshooting

Symptom	Cause & fix
`dsh: profile "tui" does not exist`	tui needs a plugin (`dsh plugin --profile tui add <pkg>`)
`npx` very slow	first-run package size; `npm i -g` helps
browser can't reach 3080	port busy: `netstat -ano | findstr 3080` → kill PID
model not responding	check `~/.dsh/settings.yaml` + API key

plugin install 404	**rc.1 dependency break**: use `^0.1.0-rc.6` line (Ch.3 pitfall #1)
behavior changed after upgrade	rc-stage breaking changes; check changelog

---

Next: Chapter 3: Profiles & the Plugin System

English | 中文 · ← Back

Chapter 3: Profiles & the Plugin System

> ***Goal of this chapter:*** *Understand dsh's customizable skeleton, how profiles are organized, how plugins are mounted, and what the host/client dual halves are.* ***This is the watershed moment from "user" to "developer."***

3.1 Profile: A Launchable Configuration Stack

dsh uses **profiles** to represent "a launchable form factor." Two are built in; the rest are created via plugins:

Profile	Purpose	Command
`web`	Web UI (conversation + sidebar + tools)	`dsh web`
`headless`	One-shot CLI tasks	`dsh --profile headless "task"`
`tui` (plugin required)	Terminal UI	`dsh --profile tui` (not built in; requires a plugin)

A profile directory looks like this (`~/.dsh/profiles/<name>/`):

```
profiles/web/
├── package.json         # Plugin dependencies + dsh.profile manifest (bundles order)
├── cordis.patch.yml     # Your patch layer: mount/override plugin declarations
├── cordis.yml           # (Generated) Composite configuration
├── pnpm-workspace.yaml
└── node_modules/
```

Loading order (from official docs): Built-in bundles (`dsh-base` → `dsh-web-app`) → profile's `cordis.patch.yml` → user-level `~/.dsh/cordis.patch.yml` → `--patch` overlay.

3.2 Mounting a Plugin: Two Changes

Here's how to mount the community plugin `dsh-better-sidebar` (a real operation):

① **Add the dependency in** `package.json` (using `link:` to point to local source, or an npm package name):

```
{
  "dependencies": {
    "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"
  }
}
```

② Add the mount line in `cordis.patch.yml`:

```
- insert:
    - id: better-sidebar
      name: dsh-better-sidebar
```

③ Install and restart:

```
cd ~/.dsh/profiles/web && pnpm install
dsh web   # Takes effect after restart
```

> 💡 *Plugins are **isolated per session** by default (e.g. `better-sidebar`'s layout/tabs persist per session). Mounting is profile-level, but state is session-level.*

3.3 Host Half & Client Half: One Package, Two Faces

A plugin can carry two runtime halves simultaneously:

Half	Runs In	Responsibility	Example
Host half	Node process	Tools, services, events, filesystem, processes	`apply(ctx)` registers tools/services
Client half	Browser (web profile)	UI, interaction, DOM	`dsh.client` declaration in `package.json` + `src/client/`

How to declare the client half in `package.json`:

```
{
  "dsh": {
    "client": {
      "inject": ["@deepseek-ai/dsh-client-runtime", "@deepseek-ai/dsh-client-locale"],
      "platform": "web"
    }
  },
  "exports": {
    ".": { "types": "./lib/types/index.d.ts", "default": "./lib/index.js" },
    "./client": { "types": "./lib/types/client/index.d.ts", "default": "./lib/client.js" }
  }
}
```

- Host half: `exports["."]` → `apply(ctx)` (the cordis plugin body)
- Client half: `exports["./client"]` → `apply(ctx)` on the browser side
- `inject`: Declares required services (cordis dependency injection)

3.4 Extension Points: Look for Hooks First, Don't Fork the Core

Official principle (stated in CONTRIBUTING/AGENTS.md): **"Plugins, not loop changes: new behavior goes on documented extension points."** The most common mistake newcomers make is modifying the core loop. The correct approach is to use extension points.

Confirmed commonly-used extension points (each explored hands-on in later chapters):

Extension Point	Location	Purpose
`agent/request` waterfall	`agent-loop`	**Modify config before every model request** (provider/model/reasoningEffort/tools). This is what tool-turbo uses.
`agent/request-error`	`agent-loop`	Intervene on request failure (the official compaction plugin uses this for context-window overflow recovery)
`conversationEvents.register`	Client runtime	Subscribe to / inject conversation events (tool/call, turn/start, etc.)
`ctx.slots.inject`	Client UI slots	Inject UI into interface slots (e.g. turnTail to display artifact file lines)
`settings` service	dsh-settings	Register user-configurable namespaces (settings page auto-renders them)
`ctx.provide` / `ctx.get`	cordis	Provide services across plugins

3.5 Common Pitfalls (Battle-Tested)

Pitfall	Symptom	Fix
rc.1 broken dependency chain	`pnpm install` reports `@deepseek-ai/dsh-type-meta@0.0.1-rc.1` 404	Several packages from the rc.1 era were never published; upgrade to the `^0.1.0-rc.6` line
Plugin missing `main`	`dsh: No "exports" main defined`	The host plugin's `package.json` must expose a `.` entry (`"main": "src/index.ts"` can be loaded directly by tsx)
Event handler forgets `await next()`	Request loses provider/model and errors out	`next()` in `agent/request` returns a **Promise**; you must await it before spreading

Type error: 'agent/request' is not assignable to keyof Events	npm types don't re-export the official type augmentations	Use a relaxed signature (ctx.on as unknown as ...) at the boundary
Client package depends on window.__ModuleLoader__	jsdom can't run client tests directly	Component-level tests need the dsh web runtime (run in official CI)

Next chapter: Chapter 4: Plugin Development, Hands-On (planned) — Write your first host plugin from scratch.

Chapter 4: Plugin Development, Hands-On

> ***Goal of this chapter:*** *Build a* ***real, working host plugin*** *from scratch, one that automatically adjusts reasoning effort via the* `agent/request` *extension point. This is a full breakdown of the community project* `dsh-tool-turbo`*. All code is runnable and testable.*

4.1 What We're Building

Problem: Before every tool call, the dsh model re-thinks from scratch (`reasoning_effort`). In a 50-step tool-chain task, "thinking" accounts for 90%+ of wall-clock time.

Solution: A host plugin that listens on the `agent/request` waterfall and downgrades `reasoning_effort` from `high` to `low` for simple turns, based on the most recent tool calls.

4.2 Project Skeleton

```
dsh-tool-turbo/
├── package.json          # Host plugin declaration
├── tsconfig.json
├── src/
│   ├── effort-decision.ts  # Pure function: decision logic (zero deps, unit-testable)
│   └── index.ts            # apply(ctx): hooks into the extension point
└── tests/
    └── effort-decision.spec.ts
```

Key fields in `package.json` :

```
{
  "name": "dsh-tool-turbo",
  "type": "module",
  "main": "src/index.ts",
  "exports": {
    ".": { "types": "./src/index.ts", "default": "./src/index.ts" }
```

```
  },
  "peerDependencies": {
    "@deepseek-ai/cordis": "^4.0.1",
    "@deepseek-ai/dsh-agent": "^0.1.0-rc.6"
  }
}
```

> ⚠️ *Always use the `^0.1.0-rc.6` dependency line. The rc.1 npm dependency chain is broken (see Chapter 3 pitfalls).*

4.3 Pure Function: Decision Logic (Zero Dependencies, Unit-Testable)

`src/effort-decision.ts`:

```
export type EffortId = 'low' | 'high' | 'max'

export interface ToolCallSample {
  name: string      // Tool name, e.g. 'write', 'read', 'bash'
  argsSize: number  // Argument size (character count)
}

export interface EffortDecisionInput {
  recentCalls: readonly ToolCallSample[]
  selected: EffortId      // User's baseline effort
  allowDowngrade: boolean
  allowUpgrade: boolean
}

const SIMPLE_TOOL_RE =
/^(fs|bash|terminal|read|write|grep|glob|edit|ls|cat|rm|cp|touch|mkdir|pwd)/i
const HEAVY_ARGS = 800

export function decideEffort(input: EffortDecisionInput): EffortId {
  const { recentCalls, selected, allowDowngrade, allowUpgrade } = input
  if (recentCalls.length === 0) return selected   // Fresh prompt: keep baseline

  const ratio = recentCalls.filter(c =>
    SIMPLE_TOOL_RE.test(c.name) && c.argsSize < HEAVY_ARGS,
  ).length / recentCalls.length
  const heaviest = recentCalls.reduce((m, c) => Math.max(m, c.argsSize), 0)

  if (ratio >= 0.75 && allowDowngrade) return 'low'
  if (heaviest >= HEAVY_ARGS * 4 && allowUpgrade) return 'max'
  if (ratio < 0.75) return allowUpgrade ? 'high' : selected
  return selected
}
```

Why extract a pure function: The decision logic is decoupled from the dsh runtime. Unit tests have zero dependencies, run in milliseconds, and cover every branch. Live verification only needs to confirm "did the injection actually happen."

4.4 Plugin Body: Hooking into the `agent/request` Waterfall

`src/index.ts`:

```
import type { Context } from '@deepseek-ai/cordis'
import { decideEffort, type ToolCallSample } from './effort-decision.ts'

export interface ToolTurboConfig {
  enabled: boolean
  allowDowngrade: boolean
  allowUpgrade: boolean
  baseline: 'low' | 'high' | 'max'
}

export const DEFAULT_CONFIG: ToolTurboConfig = {
  enabled: true, allowDowngrade: true, allowUpgrade: false, baseline: 'high',
}

const WINDOW = 8

function recentToolCalls(agent: unknown): ToolCallSample[] {
  const events = (agent as { session?: { events?: readonly unknown[] } }).session?.events ??
[]
  const out: ToolCallSample[] = []
  for (let i = events.length - 1; i >= 0 && out.length < WINDOW; i--) {
    const e = events[i] as { type?: string; data?: { name?: string; arguments?: unknown } } |
undefined
    if (e?.type !== 'tool/call') continue
    out.push({
      name: e.data?.name ?? 'tool',
      argsSize: typeof e.data?.arguments === 'string' ? e.data.arguments.length : 0,
    })
  }
  return out.reverse()
}

export function apply(ctx: Context, config: ToolTurboConfig = DEFAULT_CONFIG): void {
  if (!config.enabled) return

  // Boundary adaptation: npm package doesn't re-export official event type augmentations,
  // so we relax the signature here (see Chapter 3 pitfalls)
  const on = ctx.on as unknown as (
    event: string,
```

```
    handler: (payload: Record<string, unknown>, next: () => unknown) => unknown |
Promise<unknown>,
  ) => void

  on('agent/request', async (payload, next) => {
    const seed = await next() as { reasoningEffort?: unknown }   // ⚠️ Must await!
    const calls = recentToolCalls(payload.agent)
    const effort = decideEffort({
      recentCalls: calls,
      selected: config.baseline,
      allowDowngrade: config.allowDowngrade,
      allowUpgrade: config.allowUpgrade,
    })
    console.log(`[tool-turbo] calls=${JSON.stringify(calls)} => reasoningEffort=${effort}`)
    return { ...seed, reasoningEffort: effort }
  })
}
```

Three critical points (all learned the hard way):

1. `next()` **is a Promise:** `await next()` retrieves the current config. Spreading without awaiting yields an empty object, which drops the provider/model and causes errors.
2. **Waterfall semantics:** The listener's **return value** is passed to the next listener and ultimately to the request. Returning `{...seed, reasoningEffort}` means "keep the original config, but override the reasoning effort."
3. `agent/request` **fires every step:** The `agent-loop`'s `buildRequest` runs through this waterfall at every step, so dynamic decisions naturally take effect per-step.

4.5 Testing

Unit tests (pure function, zero dependencies):

```
import { describe, expect, it } from 'vitest'
import { decideEffort } from '../src/effort-decision.ts'

it('downgrades to low for simple tool chains', () => {
  expect(decideEffort({
    recentCalls: [{ name: 'write', argsSize: 40 }],
    selected: 'high', allowDowngrade: true, allowUpgrade: true,
  })).toBe('low')
})
// ... more branches: fresh prompt keeps baseline / downgrade disabled /
//     oversized payload upgrades to max / mixed tools upgrade to high
```

Live verification (critical, proves "the injection actually happens"):

Mount the plugin (Chapter 3 method) → restart `dsh web` → send a file-creation task → watch the dsh process logs:

```
[tool-turbo] agent/request: calls=[]                    => reasoningEffort=high
[tool-turbo] agent/request: calls=[{"name":"write",…}] => reasoningEffort=low
```

First turn has no tool calls → stays at baseline `high`. After detecting the `write` tool → next turn drops to `low`. **The injection pipeline works end to end.**

> *Full runnable code: https://github.com/Electricitysheep/dsh-tool-turbo*

4.6 Three Development Disciplines for Newcomers

1. **Find the extension point first:** 90% of behaviors you want to change have official hooks (`agent/request`, `settings`, `conversationEvents`, `slots`). Don't fork the core.
2. **Extract logic into pure functions:** Decouple decision/computation logic from dsh. Unit tests run in milliseconds and cover every branch. Live verification only needs to confirm "the injection happened."
3. **Never skip live verification:** Unit tests prove the logic; live logs prove the wiring. Both must pass before you're done.

Next chapter: Chapter 5: Real-World Cases (planned) — Git panel, HTML draft preview, speed-up plugin.

English | 中文 · ← Back

Chapter 5: Real-World Cases

> ***Goal of this chapter:*** *Walk through three* ***real, merged open-source PRs*** *to demonstrate the full plugin development loop: requirements → implementation → testing → live verification. All are genuine community PRs you can study against the source code.*

5.1 Case 1: Git Panel — Adding push / pull / fetch

Background: The community plugin `DSH-better-sidebar` provides a Git panel, but it only supports local operations (stage/commit/restore). **No remote sync.**

Implementation (4 files, all following the "pure function + routing + UI" layering):

```
src/git.ts              # Pure function layer: upstreamInfo / fetchRemote / pull / push
src/index.ts            # Routing layer: git.upstream / git.fetch / git.pull / git.push
src/client/GitView.tsx  # UI layer: upstream badge (↓behind ↑ahead) + three buttons
tests/git-sync.spec.ts  # Integration tests: full chain with a local bare repo
```

Key design (worth copying):

```ts
// push only allows --force-with-lease, never bare --force — safety red line, documented
export async function push(cwd: string, force = false): Promise<string> {
  const args = ['push']
  if (force) args.push('--force-with-lease')
```

```
  return runGit(cwd, args)
}
```

Tests (local bare repo, no network, no global git config):

```
// Covers: upstream tracking / push / pull fast-forward / fetch doesn't move HEAD /
//         force-with-lease refuses to clobber others' commits
it('force push refuses to clobber unseen remote commits', async () => {
  // Another clone pushed commits first; local rewrites history but doesn't fetch (lease is
stale)
  await expect(git.push(clone, true)).rejects.toThrow()
  // Remote still holds the other party's commits — safety guarantee holds
})
```

Live verification (Playwright driving a real dsh web instance):

- Three buttons render correctly; pull/push are properly disabled when no upstream exists
- Clicking "Fetch remote" → network request `git.fetch [200]` → panel auto-refreshes

> *PR: https://github.com/omdsh-dev/DSH-better-sidebar/pull/10*

5.2 Case 2: HTML Preview — Supporting Unsaved Drafts

Background: After editing an HTML file, the preview only shows the **saved version** (a known limitation in the README). Seems simple, but there's a security constraint: **a non-sandboxed `srcdoc` iframe inherits the parent origin** (stated explicitly in official code comments).

Solution: Extract a pure function for the "safety decision":

```
// When sandbox is on, dirty drafts use srcdoc (opaque origin, safe);
// when sandbox is off, refuse srcdoc and fall back to route-src (cross-origin guarantee)
export function htmlPreviewTarget(input): HtmlPreviewTarget {
  if (input.isHtml && input.dirty && input.draft !== null && !input.sandboxOff) {
    return { srcDoc: input.draft }
  }
  return { src: input.routeUrl }
}
```

Lesson: A UI feature may look like "just add a button," but the security model is a hard constraint. **Read the "why" in code comments first, then write code.**

> *PR: https://github.com/omdsh-dev/DSH-better-sidebar/pull/11*

5.3 Case 3: tool-turbo Speed-Up Plugin

(Fully broken down in Chapter 4; here we just summarize results)

- **Decision logic:** Pure function `decideEffort`, 6/6 unit tests passing
- **Injection pipeline:** `agent/request` waterfall; live logs confirm `high → low`

- **Value proposition** (important correction): dsh is already fast on simple tasks. **The speed-up benefit is in long tool-chain tasks** — the cumulative effect of downgrading reasoning effort at every step

5.4 Shared Methodology Across All Three Cases

1. **Pure function isolation:** Decision/computation logic has zero dependencies → unit tests cover every branch
2. **Extension point integration:** Routing/waterfall/events — never touch the core
3. **Live verification loop:** Unit tests (logic) + real dsh logs/network requests (wiring) — dual evidence

---

Next chapter: Chapter 6: Advanced & Performance Tuning (planned).

English | 中文 · ← Back

Chapter 6: Advanced & Performance Tuning

> ***Goal of this chapter:*** *Go from "it runs" to "it runs well" — reasoning effort strategy, tool-call latency analysis, and a real-world pitfall checklist.*

6.1 Performance Model: Where Does dsh Spend Its Time

Measured latency breakdown for a "create a file" task:

Phase	Share	Notes
Model thinking (Think)	~90%	**Re-thinks before every tool call** — the overwhelming majority
Tool execution	<1%	File writes and similar, millisecond-level
Network / rendering	~10%	API round-trip + UI updates

Implications:

- Simple tasks → optimize thinking time (lower reasoning effort)
- Long tool-chain tasks → save thinking time at every step; cumulative gains are largest
- Slow tools themselves (search / large files) → optimize the tool implementation, not the effort level

6.2 reasoning_effort Strategy (Official Three Tiers)

Tier	Recommended Use
`low`	Simple / deterministic turns: file operations, batch jobs, cheap steps in a tool chain
`high`	Everyday agent tasks (default)
`max`	Complex reasoning, long-chain planning, debugging

Manual: The "Reasoning Level" selector in the UI, or `reasoningEffort` in `~/.dsh/settings.yaml`.
Automatic: A plugin dynamically downgrades per tool turn (`dsh-tool-turbo`, Chapter 4).

6.3 Tool-Call Latency Visualization

dsh's session stats line (at the bottom of the Web UI) shows: `N turns • M steps | LLM Xs • Tool calls Ys | Avg first token ...` — this is the fastest way to locate bottlenecks.

Advanced: A host plugin can listen to tool events for per-tool timing (tool-turbo already includes a host-side logging version), surfacing "which tool is the slowest."

6.4 Pitfall Checklist (Battle-Tested, with Fixes)

#	Pitfall	Symptom	Fix
1	rc.1 broken dependency chain	`pnpm install` 404 (`dsh-type-meta` etc. were never published)	Use the `^0.1.0-rc.6` dependency line
2	Plugin missing `main`	`No "exports" main defined`	Expose a `.` entry; `"main": "src/index.ts"` can be loaded by tsx
3	`next()` not awaited	Provider/model lost, error thrown	`next()` in `agent/request` returns a Promise; must await
4	Event type not recognized	`'agent/request' is not assignable to keyof Events`	npm doesn't re-export type augmentations; relax the signature at the boundary
5	Client tests won't run	jsdom reports `window.__ModuleLoader__` undefined	Client artifacts depend on dsh's bootstrap mechanism; run component tests in official CI
6	Misattributing "suddenly faster" on simple tasks	1s vs 110s difference misattributed	DeepSeek context cache hits also speed things up — A/B tests must use a fresh prompt
7	Port conflict	`dsh web` won't start	`netstat -ano

6.5 Evaluation Perspective: Official Scorecards vs. Independent Benchmarks

(With the 0813 official release in mind) When reading agent model scorecards, ask three questions:

1. **Who tested it?** Official self-tests (their own harness) vs. independent third parties (AA, etc.)
2. **Which harness?** Different frameworks yield very different scores (official Terminal-Bench 87.9 vs. AA independent 79)
3. **How strict is the verifier?** Lenient verifier (SWE-bench Verified has 8.5% false positives) vs. strict (DeepSWE 0.3%)

dsh's `agent/request` waterfall makes model benchmarks reproducible. That's an engineering advantage over closed-source products.

Next chapter: Chapter 7: Ecosystem & Resources (planned).

English | 中文 · ← Back

Chapter 7: Ecosystem & Resources

> ***Goal of this chapter:*** *Give you a map for "joining the dsh ecosystem" — official entry points, the state of the community, and how to participate.*

7.1 Official Entry Points

Resource	URL	Purpose
Official repository	https://github.com/deepseek-ai/deepseek-harness	Source code, architecture docs, issues
API documentation	https://api-docs.deepseek.com	Models, pricing, API guides
Discord	Link in official README	Community discussion
Discussions	Official repo Discussions	Proposals / help requests (currently the recommended contribution entry point)

Note: The official CONTRIBUTING guide states "external PRs are not accepted at this time" (as of 2026-08-13). However, the following are encouraged:

- Submit suggestions in Discussions (the team evaluates them)
- **Build dsh-plugin ecosystem projects** (an officially endorsed contribution path)
- Write tutorials and blog posts

7.2 Plugin Ecosystem (Snapshot as of 2026-08-13)

Project	Focus	Status
`DSH-better-sidebar`	File management / terminal / Git / browser sidebar	Most complete community plugin
`dsh-tool-turbo`	Tool-call speed-up (automatic reasoning_effort adjustment)	Community speed-up plugin
`dsh-handbook` (this handbook)	Beginner tutorial	Ecosystem documentation

Discovering plugins: Search GitHub for `topic:dsh-plugin`. **Publishing a plugin:** Add the `dsh-plugin` topic to your repo + publish on npm.

7.3 How to Join the Ecosystem (Beginner Path)

1. **Use it:** `dsh web` + install two community plugins, integrate into your daily workflow
2. **Small improvements:** Submit PRs to community plugins (read the three cases in Chapter 5 for a complete PR paradigm)
3. **Publish a plugin:** Start from the minimal host plugin in Chapter 4, tag it with `dsh-plugin`
4. **Write content:** Tutorials / reviews / pitfall guides (officially encouraged); cross-reference this handbook

7.4 Recommended Reading Path

Goal	Path
Quick start	Chapter 2 → install better-sidebar → use daily
Plugin development	Chapter 3 → Chapter 4 → follow the Chapter 5 cases
Performance tuning	Chapter 6 → tool-turbo source code
Deep customization	Official AGENTS.md (architecture) → docs/architecture.md → packages/ source code

Closing Words

dsh was open-sourced on 2026-08-13. **Every day of the ecosystem is "early days."** This handbook will keep updating as dsh evolves. If a command in some chapter stops working, it's most likely due to rc version iteration. Always defer to the official changelog.

Go claim your spot in this brand-new ecosystem. 🚀

English | 中文 · ← Back

Chapter 8: Tools & Context System

> ***Goal of this chapter:*** *Understand dsh's "capability engine" — which tools the model can call, how context is fed to the model, and how long conversations are handled.* ***This is the key chapter for going from "it runs" to "I understand how it runs."***

8.1 Official Capability Map (60+ Packages at a Glance)

All of dsh's capabilities are provided as packages (`packages/<group>/<name>`). The ones newcomers need to know first:

Capability Domain	Official Packages	Purpose

Tools	`fs/tool-fs`, `fs/tool-fs-search`, `fs/tool-str-replace-editor`, `shell/tool-bash`, `web`, `skill`, `todo`	Callable tools for files, terminal, web, skills, todos, etc.
Context	`context/*`, `compaction/*`	Request context assembly, long-conversation compression
Session	`session/*`	Persistence, titles, telemetry
Subagent	`subagent/*`	Delegate sub-tasks
MCP	`mcp/*`	MCP client (external tool servers)
Workflow	`workflow/*`	Multi-step workflow orchestration
Safety	`sandbox/*`, `guard/*`, `interaction/*`	Sandboxing, loop hygiene, permissions/approvals
Model (LLM)	`llm/*`, `llm-deepseek`, `llm-retry`	Model integration, retries
Skill	`skill/*`	Skill provider registry
Client (UI)	`client/*` (ui-conversation, ui-tool, ...)	Web UI components

> *Full list: see `packages/AGENTS.md` in the official repository.*

8.2 Built-In Tools (Observed in Practice)

The actual tool names the model can call are **short verbs** (observed from dsh web sessions and agent/request logs):

Tool Name	Purpose	Notes
`read`	Read files	fs capability
`write`	Write files	fs capability
`grep`	Content search	fs-search
`glob`	File pattern matching	fs-search
`edit` / `str_replace_editor`	Precise editing	Tool results include `locations` (used for artifact tracking)
`bash` / `pwsh`	Execute commands	Sandbox isolation
`todo`	Todo management	Long-task planning
`skill`	Skill invocation	Injected via skill-catalog

Tool results and artifact tracking (important concept): Tool return values carry `locations` (file paths). dsh uses these to build "artifact file lines" — the artifact chips at the end of a conversation

come from here. **The model can see which files were changed, and the UI lets you open them directly.**

8.3 How Context Is Fed to the Model

A single model request's context = system prompt + skill catalog + conversation history + tool results. This is visible in session logs:

```
Context injection @deepseek-ai/dsh-system-prompt   ← Official system prompt
Context injection skill-catalog                    ← Skill catalog
```

dsh's context mechanism:

- **Layered system prompts:** Official plugins register prompt sections via `systemPrompt.section()` (e.g. ui-deliverables registers "artifact file reference" guidance)
- **Skill catalog injection:** The list of available skills enters the context; the model calls them as needed
- **Tool schemas:** Every request carries tool definitions

8.4 Long Conversations: Compaction

Long conversations blow up the context window. dsh's `compaction` plugins (e.g. `compaction-basic`) handle:

- Detecting context overflow (`CONTEXT_WINDOW_EXCEEDED` via `agent/request-error`)
- Compressing history (model-agnostic pruning + optional summarization)
- Routing to an "overflow agent" on failure

> *For newcomers: **Just know that "long conversations are auto-compressed."** The details are an advanced topic. In production, note that compression loses detail — important context should be written into the prompt manually.*

8.5 Permissions & Security Model (Awareness Level)

- **Access modes:** The UI shows modes like "Workspace Write" (permission presets)
- **Interactive approvals:** `interaction/*` provides permission/approval capabilities (dangerous operations can require confirmation)
- **Sandboxing:** `sandbox/*` isolates command execution (e.g. the pwsh sandbox has ACL constraints — temp directory permission issues have been observed in practice)
- **Tool timeouts:** `guard/*` provides loop hygiene and tool timeouts

> *Deep security configuration is beyond the scope of this handbook. The key takeaway: **dsh's tool execution has isolation and approval layers by default.** It's not bare execution.*

8.6 Three Things Newcomers Should Remember Most

1. **Tool names are short verbs** (read/write/grep/glob/edit/bash) — when writing prompts or plugins, just say "read the file" or "search" and it works
2. **Tool returns include** `locations` → **artifact tracking** — files the model changed appear in the conversation's artifact area

3. **Long conversations are auto-compressed** — no need to manually clear history (but important info should go into the prompt)

Next chapter: Chapter 9: MCP, Subagents & Workflows

English | 中文 · ← Back

Chapter 9: MCP, Subagents & Workflows

> ***Goal of this chapter:*** *Understand dsh's three extension capabilities — MCP (connecting external tools), subagents (parallel tasks), and workflows (multi-step orchestration).* ***These are the switches that upgrade dsh from a "single agent" to an "agent system."***

> ⚠️ *This chapter is based on official architecture docs (`packages/AGENTS.md`) and package structure. Some feature examples are marked "pending live testing" — rc versions iterate quickly, so defer to the official changelog for exact usage.*

9.1 MCP: Connecting to the External Tool Ecosystem

MCP (Model Context Protocol) is an open protocol for "plugging external tools into an agent." dsh provides an `mcp` client package (`@deepseek-ai/dsh-mcp-client`).

What it does: Connect to any MCP server via MCP (databases, browsers, charts, internal company systems, etc.). For example, the community's `dsh-plugin-cost-tracker` (token tracking) is an MCP/plugin hybrid.

Positioning for newcomers: MCP is an "ecosystem extension." Only reach for it once you have a clear tool gap inside dsh. Get the built-in tools working first, then add MCP.

9.2 Subagents: Working in Parallel

What they are: Delegate tasks to sub-agents for parallel execution (`packages/subagent/*`).

Typical scenarios:

- Multi-module research in a large repo (one subagent per module)
- Long-task decomposition (parent agent plans, subagents execute)
- Independent verification (subagents cross-check each other)

For newcomers: Subagents are "advanced weaponry." Master the single-agent workflow first, understand task decomposition, then move to subagents. A common anti-pattern is "spawning a subagent for everything," which just adds coordination overhead.

9.3 Workflows: Multi-Step Orchestration

What they are: `packages/workflow/*` provides multi-step workflow orchestration (worker-thread provider + tool Consumer).

How they differ from "multi-turn conversation": A normal conversation is "the model freestyles." A workflow is "steps are defined as a flow, executed in order or by condition." This suits

deterministic processes (e.g. fetch data → clean → generate report → validate).

Official examples (`packages/examples/`): Runnable cordis.yml workflow samples are available.

9.4 Putting It Together: What an "Agent System" Looks Like

```
Your prompt (goal)
   ↓
Parent Agent (planning)
   ├─── Subagent A: Research module A (parallel)
   ├─── Subagent B: Research module B (parallel)
   └─── Tool chain: read/grep/bash + MCP (database)
   ↓
Workflow (e.g. validate → aggregate → output report)
   ↓
Artifacts (openable / trackable)
```

Suggested beginner path:

1. **Stage 1:** Single agent + built-in tools (Chapters 2–5)
2. **Stage 2:** + MCP (connect external tools)
3. **Stage 3:** + Subagents (parallelism)
4. **Stage 4:** + Workflows (deterministic flows)

9.5 Community Ecosystem Examples (Snapshot as of 2026-08-13)

Community projects already appearing in the official Discussions:

- `dsh-plugin-cost-tracker` — real-time token cost tracking (plugin/MCP hybrid)
- Plugin development / speed-up tools (e.g. `dsh-tool-turbo` , the companion to this handbook)

The ecosystem is growing fast. **Now is a great time to start building.**

Appendix A: Glossary & Command Quick Reference

English | 中文 · ← Back

Chapter 10: Complex Real-World Cases (Run Live in dsh)

> ***Goal of this chapter:*** *Demonstrate dsh's actual capabilities, output quality, and timing on multi-step tool chains through two* ***complex tasks completed live in dsh. Privacy notice:*** *Both cases use* ***synthetic data / self-authored code*** *only. No real business data or sensitive information is involved.*

Case A: Data Quality Analysis + Cleaning + Visualization (186 seconds)

Task

Give dsh a synthetic JSON file with dirty data (52 rows: missing values, type errors, duplicate rows, outlier value 99999), and ask it to:

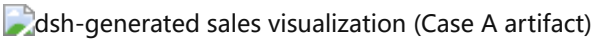
1. Analyze data quality issues
2. Write a cleaning script `clean.py`
3. Write a visualization script `visualize.py` (generating `chart.png`)
4. Run both scripts to verify

How dsh Did It (Tool Chain)

```
read(sales_data.json) → write(clean.py) → bash(python clean.py)
→ write(visualize.py) → bash(python visualize.py) → read(output) → summary
```

Output & Verification

Artifact	Result
`clean.py`	✅ Runs successfully, 52 → **35 rows**, all issues resolved
`visualize.py` + `chart.png`	✅ Valid PNG generated (1950×825)
Cleaning strategy	Median imputation for missing amounts, outlier removal, deduplication, type normalization


dsh-generated sales visualization (Case A artifact)

Output Highlights (Shows Judgment, Not Just Mechanical Execution)

> - *Median imputation creates a 510 spike in the histogram — an inherent side effect of imputation, worth noting*
> - *If 99999 represents a genuine large order, consider IQR/quantile methods instead of blanket deletion*
> - *Post-cleaning stats: N=35, mean 489.43, median 510, std dev 181.73*

dsh didn't just execute the task — it proactively explained the trade-offs and assumptions in data processing. This is evidence of "thinking" at the agent engineering layer.

Case B: 5-Bug Code Fix + 49 Tests (94 seconds)

Task

Give dsh a Python calculator module **deliberately seeded with 5 bugs**, and ask it to: find all bugs → fix them → write comprehensive unit tests → run and verify.

The Planted Bugs

1. `add` mistakenly written as `a - b`
2. `divide` silently returns 0 on division by zero (should throw)
3. `factorial` silently returns -1 for negative input (should throw)
4. `factorial` 's `range(1, n)` misses multiplying by n
5. `format_money` doesn't format to two decimal places

dsh's Fixes + Tests (Verification: 49 passed)

Full test file: case-test-calculator.py — 49 test cases covering:

Function	Coverage
`add`	Positive / negative / zero / float / precision / commutativity
`subtract`	Negative results / zero boundary / float
`multiply`	Sign combinations / multiply by zero / float
`divide`	Integer division / signs / float / **division by zero must throw ZeroDivisionError**
`factorial`	0!/1! boundary / known values / **negative must throw ValueError**
`format_money`	Zero-padding / rounding / **0.1+0.2 rounding** / negative

```
python -m pytest test_calculator.py -v   # 49 passed in 0.37s
```

Observations: dsh's Performance

- **Full closed loop:** Read → fix → write tests → run tests → summarize, with no human intervention
- **Fix quality:** All 5 bugs fixed, with docstring explanations added
- **Test design:** Covers edge cases (division by zero / negatives / precision), not just "does it run"

Case Summary: dsh's Profile on Complex Tasks

Dimension	Performance
Multi-step tool chain	✅ Auto-orchestrated (read → write → run → verify → summarize)
Complex task timing	94s–186s (same model, same gateway; slower than benchmark simple tasks but manageable)
Output quality	✅ Shows judgment (data trade-off explanations, test edge-case design)
Artifact tracking	✅ Generated files can be opened directly at the end of the conversation
Failure recovery	Not triggered in these cases; production recommendation: pair with guard timeouts + human approval

Takeaway for newcomers: dsh excels at tasks involving "multiple files, multiple steps, and verification" — which is also its primary value proposition as an agent runtime. For complex tasks, the recommendation is to **write acceptance criteria clearly in the prompt** (e.g. "run to verify"), and dsh will follow through to completion.

Appendix A: Glossary & Command Quick Reference